第 3 章　怎样才能让机器做加法

加紧干吧，还有什么能比证明你是个天才更重要？

现在，我们已经拥有了二进制的知识，也学会了用开关将电流表示成二进制比特，为了欣赏到这些比特，我们还使用了灯泡。看起来开端不错，现在的核心工作就是搞清楚这个加法机的内部构造。这不是一件容易的事儿，我们需要好好规划一下，毕竟不能指望吹口仙气就能完成这项杰作。

在开始之前，我想大家都会同意这样的观点，那就是用机器来算数学题所面临的主要问题在于，没有生命的东西，比如一截电线、一块铜皮，是缺乏智慧、不会算数学题的。为了制造一个会算数学题的机器，必须先回顾一下我们自己算数学题的方法和过程，然后用零件和电路来模拟我们的过程。

理解了这一点，那就让我们先从熟悉的十进制加法开始，然后再接着看一看如何用二进制数做加法，但愿能得到一些启示。祝我们好运。

3.1　我们是怎样用十进制做加法的

通常，我们说“加法”的时候，都是指十进制加法。可以非常肯定地说，读这本书的人全都会做加法。这是毫无疑问的，要不然的话你简直无法生活：你不能做买卖，因为你不知道人家买的东西加起来一共是多少钱；你也不能请客吃饭，因为你不知道需要为来的客人准备多少饭菜，更不知道该在桌子上摆放多少碗筷。想到这里，我觉得，哎呀，好在我们从小就学会了这项技能，真是幸运。

尽管做十进制加法很容易，完全是不假思索地在干这件事儿，但那只是因为我们每天都在使用它，对它实在是太熟悉了，完全忘了做加法其实是一个运用口诀的过程。十进制加法的计算口诀是

0 加 0 等于 0;
0 加 1 等于 1;
0 加 2 等于 2;
0 加 3 等于 3;
⋮
9 加 7 等于 6，进 1;

9加8等于7，进1；
9加9等于8，进1；

嗯，还真是挺复杂的，要知道，为了节省篇幅，已经省略了它们中间的绝大部分。下面通过一个例子，比如计算15+7，来看一看这些口诀是如何运用的。

如图3.1所示，在实际做加法的时候，要将被加数和加数从右边对齐，而开始计算的时候，也同样是从最右边的那一列开始。

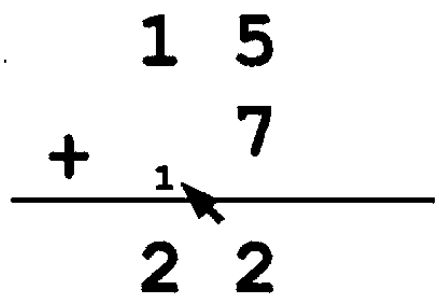

图3.1 十进制加法示意图

首先是5+7。按照口诀“5加7等于2，进1”的指示，得到结果“2”，并向左产生一个进位。为了防止过后把这个进位忘了，上学的时候老师会告诉我们一个小小的技巧，在左边一列的下面写一个小小的“1”，表明这只是个进位。

现在左边只剩下“1”了，因为这一列只有它自己，所以通常是直接拽下来，作为结果的一部分。但是别忘了，这一列上还趴着一个进位，正眼巴巴地等着被“加”一下。所以，我们还要再使用口诀“1加1等于2”来得到这一列的结果“2”。至此，我们已经完成了这道加法题，结果是22。

虽然每个人都会做加法，但是有时候，拥有这样的本领也不容易，常常需要付出遭受两面夹击、担惊受怕的代价。我二哥就是这样，经常让我妈拿着棍子往学校里撵，到了学校之后还得像受惊的兔子一样生怕遭到老师训斥。就这样年复一年，不知道是啥时候，他居然也就会了（哎呀，现在说我二哥的坏话，希望回去以后他不会拿着棍子撵我才好）。

3.2 用二进制做加法其实更简单

尽管讨论十进制加法对我们来说显得很轻松，但我们不应该停留在这里，毕竟它不是这一章的重点。现在，让我们接着讨论另外一个更重要的话题，它关系着我们的加法机是不是能顺利地造出来。

也许你已经知道（或者想到），二进制数也可以像十进制数那样进行各种数学运算，比如：

$$11101 + 110 = ?$$

尽管已经学习过二进制，但上面这道题对我们来说还是像天书。不要紧，我

来给大家翻译一下。因为 11101 和 110 分别等于十进制数的 29 与 6，所以这道题其实算的是

29 + 6 =?

当然了，结果应当是 100011，即十进制数 35。所以，在数学公理面前，不管你说的是外国话，还是中国话，也不管你用的是什么数制，都不重要。

我知道你这会儿在想什么。你一定觉得奇怪："我就纳闷儿了，两个二进制数相加，就得到了另外一个二进制数，这是怎么得到的呢？"

我们知道，在做十进制加法的过程中有两点是比较重要的：一是必须掌握加法口诀；二是需要考虑进位。很不巧，如果要做二进制加法，这两点同样很重要，简直是避不开。

加法口诀的作用是让你念念有词地得出结果。前面已经说过，十进制加法有一大堆口诀，之所以有这么多，是因为十进制有 0～9 这十个基本数字，这百十来条口诀就是用这十个基本数字翻来覆去组合而成的。

相比之下，二进制加法的口诀会比较简捷，因为它只有两个基本数字：0 和 1。所以，口诀的第一句是

0 加 0 等于 0

搞科学技术的，都要费力劳神，兢兢业业才能出点成果，就像是养育孩子。所以，要是有些学科跟养孩子能扯上关系，也就不那么奇怪了。考虑到这些，上面那句口诀的意思可以理解为

爹不揍，妈也不揍，孩子不痛（上午不痛，下午不痛，一直不痛）

父母打孩子，生活中时有所见。要不要打孩子，该不该打孩子，学术界的意见还不是很统一。有人认为父母要和孩子做朋友，以说服教育为主；也有人觉得 6 岁左右适当给点教训，有助于孩子成长。我们不是教育家，也不知道哪种观点更有道理。但显而易见的是，爹揍妈不揍，或者爹不揍妈揍，孩子都会痛的：

0 加 1 等于 1;

1 加 0 等于 1

不要被表面现象所迷惑，尽管十进制里也有 0 和 1 这两个数字，但前面这三条不是十进制加法，尽管站在十进制的角度来看它们毫无疑问也是正确的。

0 和 1 可以有 4 种相加的组合形式，前面已经有了 3 种，还剩下最后一种，即"1"和"1"相加。现在，二进制加法和十进制加法的相似性已经到此结束，在二进制里，1+1=10，但加法口诀的规则是必须表示成进位的形式：

1 加 1 等于 0，进 1

没啥说的，因为孩子做了天大的错事——比如把父母的结婚纪念照拿去做风筝，父母都气昏了头，于是一齐动手开打。结果打得太狠，孩子当时不觉得痛（木了），到了下午或者第二天，就开始痛了。

打孩子是不对的，上面这 4 条，就是二进制加法口诀的全部内容，我们不妨称之为“第一亲子定律”，让那些将来要做父母的人不但学了计算机技术，还时刻保持警醒，懂得要爱护自己的孩子。

学习了二进制加法的口诀之后，现在，让我们实际做一下，操练操练。这回来个简单的，能说明问题即可，比如 110 + 11。以十进制的眼光来看，它实际上计算的是 6 + 3。

如图 3.2 所示，和十进制加法一样，两个要加起来的数先是右对齐，然后从最右边的列开始计算。

```
      1 1 0
+       1 1
  1   1
-----------
  1 0 0 1
```

图 3.2　二进制加法示意图

先是“0”加上“1”，这正好对应于口诀“0 加 1 等于 1”，所以这一列的和是“1”；

接着，是两个“1”相加，依据口诀“1 加 1 等于 0，进 1”，所以这一列的结果是“0”，同时向左边产生一个进位“1”；

继续往左边来，这一列是个光杆司令，但不能直接拽下来，因为底下还趴着一个进位“1”。依据“1 加 1 等于 0，进 1”，这一列的结果又是“0”，而且也向左边扔出个进位。但是，左边已经什么都没有了，这才是名副其实的光杆司令，那就直接把它拽下来吧。所以，最终的结果就是 1001。

至此，我们已经得出 110+11=1001。这个结果是不是正确呢？好办，翻到第 2 章，你就会发现，二进制数 1001 确实等于十进制数 9。如果你还不放心的话，可以多找一些二进制数来试试。

3.3　使用全加器来构造加法机

在学习了二进制加法运算之后，大家可能会感觉很愉快、很满足（通常吃饱了之后也是这种感觉，这证明知识的确也是食粮）。高兴之余，你可能想在别人——比如你的父亲面前卖弄一下你的学问，向他演示 101+11=1000。看到你拥有这样非凡的才能，他肯定表现得非常激动：“揍死你这个不成器的东西，竟然连加法都不会做了！”[1]这也难怪，大人们很少有懂得二进制的。如果不是为了研制计算机，

① 在这本书中，我可能不止一次地提到普天之下的父亲，以及他们的严厉。不用担心，他也许只是吓唬吓唬你。

我们干吗要研究二进制，和它纠缠在一起？

任何一个二进制数都是由一个以上的比特组成的，是一个比特串。为了突出组成它的每个比特，一个二进制数可以表示成（如果它包含了 6 比特的话）：

$$a_5a_4a_3a_2a_1a_0$$

在这里，a_0、$a_1 \cdots a_5$ 都是单个的比特，是这个二进制数的每一位。奇怪吧？它们都有一个“a”，脚底下还吊着一个独一无二的下标。什么意思呢？“a”表示它们都属于同一个大家庭，即这个二进制数；下标则指明了它们在这个二进制数中的位置，或者说，在这个大家庭里的长幼次序。

我们在日常生活中通常都是从 1 开始编号，但是这里却是从 0 开始，而且从右向左递增。为什么要这样干呢？电子计算机是外国人发明的，我们研究计算机技术，引进他们的技术资料，自然也就把这个学来了。交朋友要交脾气相投的，干事业要找志趣一样的。要是你和自己的创业伙伴一起放羊，晚上赶羊进圈的时候，你数的是 1、2、3、4、5，共 5 只，而他数的是 0、1、2、3、4，共 4 只，弄不好你们俩就要反目。

所以，如果 $a_5a_4a_3a_2a_1a_0$=110101，那么它们的对应关系就如图 3.3 所示。

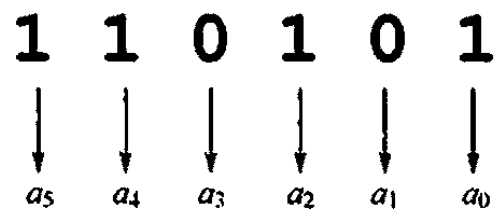

图 3.3　二进制数中各个比特的编号方法

你可能怀疑我的动机，正在猜想我为什么要跟你说这些。你的怀疑是对的，因为我正是想和你一起分析分析，在不知道或不用知道两个二进制数究竟是几的情况下，它们是如何相加的，这里面都有些什么规律。如果可能的话，我们将会得到一份奖赏，那就是顺利地把期盼已久的加法机造出来。

既然知道了我的心思，那么，让我们来看看，随便两个二进制数相加时会怎样，比如：

$$a_5a_4a_3a_2a_1a_0 + b_5b_4b_3b_2b_1b_0$$

它们可以是任意两个数。照惯例，要先把它们右对齐，叠在一起，如图 3.4 所示。

$$\begin{array}{cccccc} & a_5 & a_4 & a_3 & a_2 & a_1 & a_0 \\ + & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\ \hline \end{array}$$

图 3.4　两个二进制数相加、要和十进制加法一样对齐

因为最先相加的是最右边那一列，即 a_0 和 b_0，所以这里没有从其他列来的进

位，属于单纯的两个比特相加。如图 3.5 所示，这里有 4 种可能的情况。

```
 0      0      1      1
 0      1      0      1
 ---    ---    ---    ---
 0      1      0      0
```

图 3.5　不考虑其他列的进位时两个比特相加的 4 种可能

依据二进制加法口诀，很显然，只有在来自被加数和加数的比特都为“1”的时候，也只有在这个时候，才会向左边甩出一个进位“1”，并且结果是“0”。

相比之下，倒数第二列就没有这么幸运了。因为它不单纯是 a_1 和 b_1 相加，还可能会收到最右边那列赠予的礼物——进位，这样这一列实际上就是三个比特相加。当然，要是运气好，最右边一列没有产生进位，那么情况会和图 3.5 一样。

不过，如果最右边一列产生了进位，那么这一列实际上是另外 4 种可能，如图 3.6 所示。

```
 0      0      1      1
 0      1      0      1
 1      1      1      1
 ---    ---    ---    ---
 1      0      0      1
```

图 3.6　存在其他列的进位时两个比特相加的 4 种可能

综合图 3.5 和图 3.6，就得到了在这一列上做加法时，所有可能出现的情形，共 8 种。我想大家都看得很清楚，这一列相加的结果可能是 0，也可能是 1。而且，也可能产生进位，但这个进位必定是 1（而不会是其他数值）。

我们可以接着研究其他列上相加的情况，但实际上已无必要，原因是除了最右边那一列，不管哪一列相加，情况都和刚才讨论的一样，每一列都有可能需要加上前一列来的进位（1），相加的结果可能是 0，也可能是 1，并且它自己也可能会把一个进位（1）塞给别人。

既然是这样，你的大脑也许会闪出灵感的火花，迸出一个非常绝妙的想法：既然加法都是按列进行的，而且每一列的计算过程都一样，那么完全可以设计一个电路来完成每一列的相加过程，如图 3.7 所示。

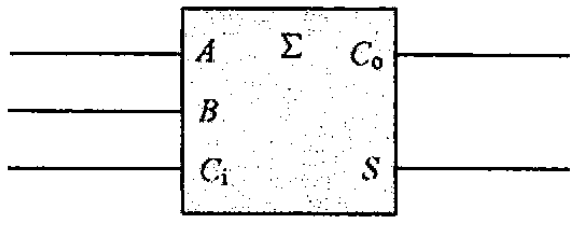

图 3.7　全加器示意图

在图中，A 和 B 分别是来自被加数和加数的一个比特，它们正好在同一列上；C_i 是来自右边一列的进位；C_o 是本列产生的进位；S 是本列的“和”。为了表明这个电路的用途，我们在图的中间加了一个符号“Σ”。在数学中，这个符号用来表示“加”，它的读音是“西格马”。很显然，这并不是马的一个新品种。

既然是一个电路，它肯定有一个名字。是的，它叫全加器。这不是一个很容易理解的名字，特别是这个“全”字。而且，既然有全加器，是不是还应该有“半加器”？你别说，还真有半加器这东西。但是，半加器仅仅是把来自被加数和加数的两个比特加起来，产生一个“和”以及一个进位，并不考虑从其他列来的进位。换句话说，它只是用电路来实现二进制加法口诀。全加器则不然，它真正实现了二进制加法中每一列的加法过程，所以它才叫做“全加器”。

有了全加器，解决了二进制加法过程中每一列的计算问题，那么，我们可以搞一大堆全加器，根据被加数和加数的比特数，把它们串联起来组成一个完整的加法电路，图 3.8 显示了这一过程。

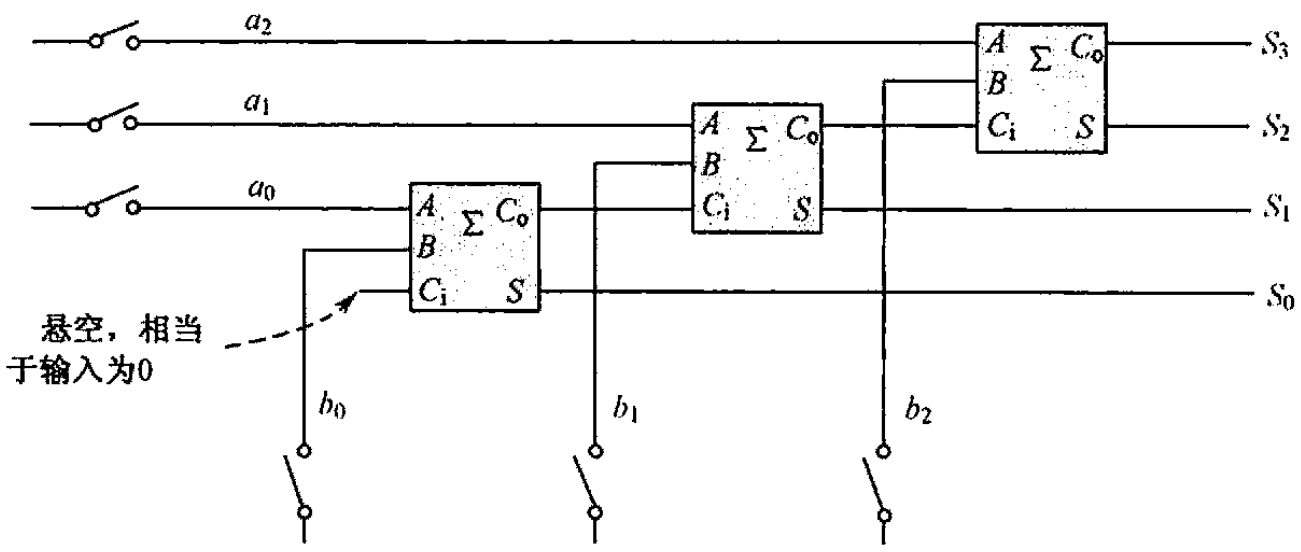

图 3.8　由 3 个全加器组成的 3 比特加法机（结果是 4 比特）

图中，参与相加的两个二进制数分别是 $a_2a_1a_0$ 和 $b_2b_1b_0$，组成它们的每一个比特都可以用开关的闭合与断开来得到。

随着开关的闭合与断开，我们会得到一些二进制数，比如 $a_2a_1a_0$=110，这个很正常。

但是，也有可能会得到一些看似不那么正常的，比如 $a_2a_1a_0$=011。不要大惊小怪，这其实很正常，它实际上就是 11，只不过前面多了个“0”。和十进制一样，在 1 左边不管有多少个“0”，都不会改变它的大小。有人不是说嘛，“1”就好比是健康，“0”就是财富。健康打头，财富多了才有意义；否则，你把健康放到最后，前面的财富再多有什么用呢？

因为被加数和加数各自用了 3 个开关，很明显，参与运算的被加数和加数都只能是 3 比特的二进制数，比如 110+101。之所以没有使用比 3 比特更大的二进制数，是因为较小的数能够使我少画一些电路，这样看起来更清楚，并因此而节

约了篇幅（印刷之前，出版社的老编们可能会重新绘制这些图，我希望他们能感谢我如此善解人意，并在适当的时候请我吃饭）。

像按列做加法一样，3 个全加器串联在一起，把被加数和加数中位置相同的两个比特相加，输出结果，并将进位传递给下一个全加器。可以看出，第一个（左下角那个）全加器的进位输入端没有使用，意思是“没有进位输入”，或者“从后面来的进位是 0”。其余的全加器，它们的进位输入端都和前一个全加器的进位输出端相连，意思是“前面的，你产生进位了吗？如果有，我得加上它”。$S_2S_1S_0$是两个二进制数相加后的最终结果。S_3 是最后一个全加器产生的进位，由于这是最后一个全加器，所以它的进位也是最终结果的一部分。尽管 100+1 的结果是 3 比特的 101，但是 100+100 却产生 4 比特的结果 1000，最左边的那个“1”纯粹是由于进位产生的。

如果仔细品味的话，你还可以发现它很容易进行扩充以计算更大的数（这样它会更实用），唯一所要做的就是在现有的基础上再串联更多的全加器。现在你应该感到高兴，看看你自己，进步多大呀！刚拿起这本书的时候，你对电子技术还知之甚少，现在，你连这样复杂的电路都能看懂了，这难道不值得庆贺吗？

在所有的计数方法里，二进制不见得是最实用的，它不够简明，写起来麻烦。别人不敢说，我相信交警们一定喜欢十进制，而不是二进制。当他们在路上执勤的时候，肯定能迅速记住刚才超速的是“计 A9998”，而“计 A10011100001110”这样的车牌号只能让他们眼晕。

好吧，我承认大家平时过日子不需要它。但是，计算机需要它。很奇怪吧？就因为它简单。它的全部，它的一切，就只有 0 和 1。简单意味着具有较少的运算规则；较少的运算规则意味着设计不太复杂；不复杂的设计又意味着可以用很少的材料来制造，并最终节省人力、物力、时间和金钱，还能保证机器工作的可靠性。

要造一台机器来计算加法，全加器无疑是最基础的零部件了。但是，这个全加器到底应当如何构造呢？看起来，我们已经离目标不远了，应该很快就能揭开最终的谜底。

事实上，还差得远呢。要想知道全加器的内部都是些什么，它是如何把 3 个比特加起来的，我们还必须回到过去，了解电与磁的历史，以及先哲们创立的逻辑学，看能不能从中得到一些启发。